



26 de septiembre de 2026

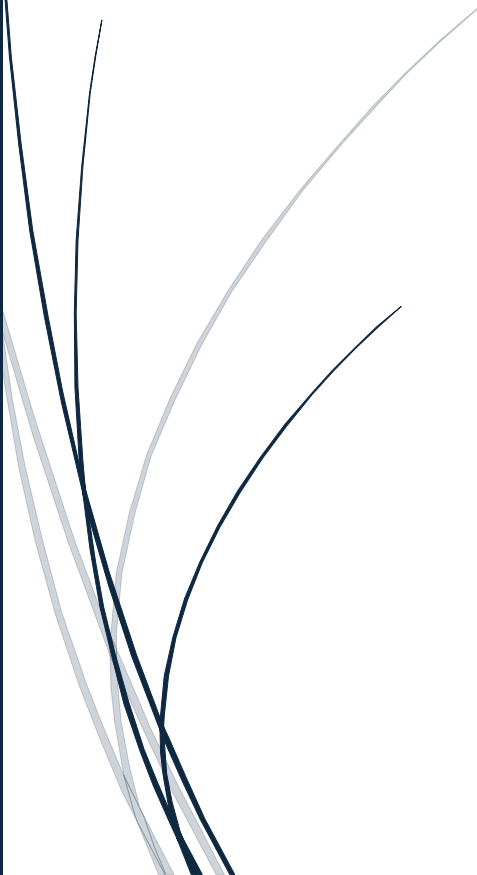
Lab: File path traversal, validation of file extension with null byte bypass

CNO IV – Seguridad Informática

Profesor: Servando López Contreras
Universidad Politécnica de San Luis Potosí

Luis Felipe Jaramillo Barajas

MATRICULA: 183559



Contenido

1. Introducción	2
2. Objetivos	2
Objetivo general	2
Objetivos específicos.....	2
3. Preparación del laboratorio	3
4. Fase 1: Interceptación y confirmación de la validación de extensión	3
5. Fase 2: Construcción del payload con byte nulo	5
6. Fase 3: Explotación y confirmación	6
6.1 Explicación del payload	6
6.2 Resolución del laboratorio	7
7. Mitigación recomendada.....	8
8. Conclusiones	8

1. Introducción

Este informe documenta el último de la serie de laboratorios de path traversal abordados en este portafolio. En este caso, además de comprobar el contenido del nombre de archivo, verifica que dicho nombre termine con la extensión esperada.

Esta validación, aunque parece razonable, puede evadirse insertando un byte nulo (%00) entre la ruta de traversal y la extensión exigida, ya que muchas funciones de bajo nivel del sistema operativo interpretan el byte nulo como el fin de la cadena, ignorando todo lo que le sigue, incluida la extensión que la aplicación cree haber verificado.

2. Objetivos

Objetivo general

Demostrar la capacidad de evadir una validación de extensión de archivo mediante la inyección de un byte nulo, aprovechando la discrepancia entre la validación a nivel de aplicación y la interpretación de cadenas a nivel de sistema operativo.

Objetivos específicos

- Confirmar que la aplicación exige que el nombre de archivo termine en una extensión concreta (.png).
- Comprender el concepto de byte nulo (null byte) y su efecto en cadenas de tipo C.
- Construir un payload que combine una secuencia de traversal con un byte nulo codificado y la extensión exigida.
- Acceder al contenido del archivo /etc/passwd evadiendo la validación de extensión.
- Proponer medidas de mitigación adecuadas.

3. Preparación del laboratorio

La aplicación valida que el nombre de archivo proporcionado termine con la extensión esperada (.png) antes de servirlo. El objetivo es recuperar el contenido de /etc/passwd.

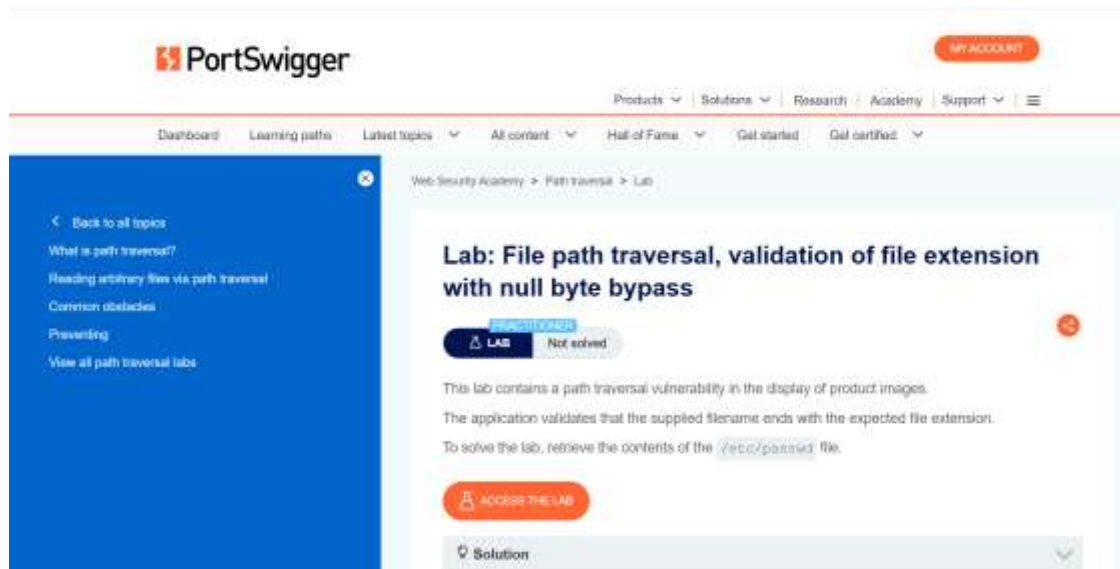


Figura 1. Descripción del laboratorio “Validation of file extension with null byte bypass” en PortSwigger Web Security Academy.

Concepto teórico: el byte nulo y las cadenas terminadas en cero. En lenguajes como C, y en muchas de las bibliotecas del sistema operativo escritas en C que subyacen a funciones de manejo de archivos, una cadena de texto se considera terminada en el primer byte con valor 0x00 (el “byte nulo”), sin importar qué caracteres existan después de él en memoria. Si una aplicación de alto nivel (por ejemplo, en Java o Python antiguo) valida la cadena completa recibida del usuario, pero luego delega la apertura del archivo a una función de bajo nivel que trunca la cadena en el primer byte nulo, es posible que ambas capas “vean” cadenas distintas: la capa de validación ve el nombre completo con la extensión correcta al final, mientras que la capa de sistema de archivos ve solo la parte anterior al byte nulo.

4. Fase 1: Interceptación y confirmación de la validación de extensión

Con el proxy activo, se navegó por el catálogo de “We Like To Shop”, confirmando el patrón habitual de peticiones GET /image?filename=N.jpg para cargar cada imagen de producto.

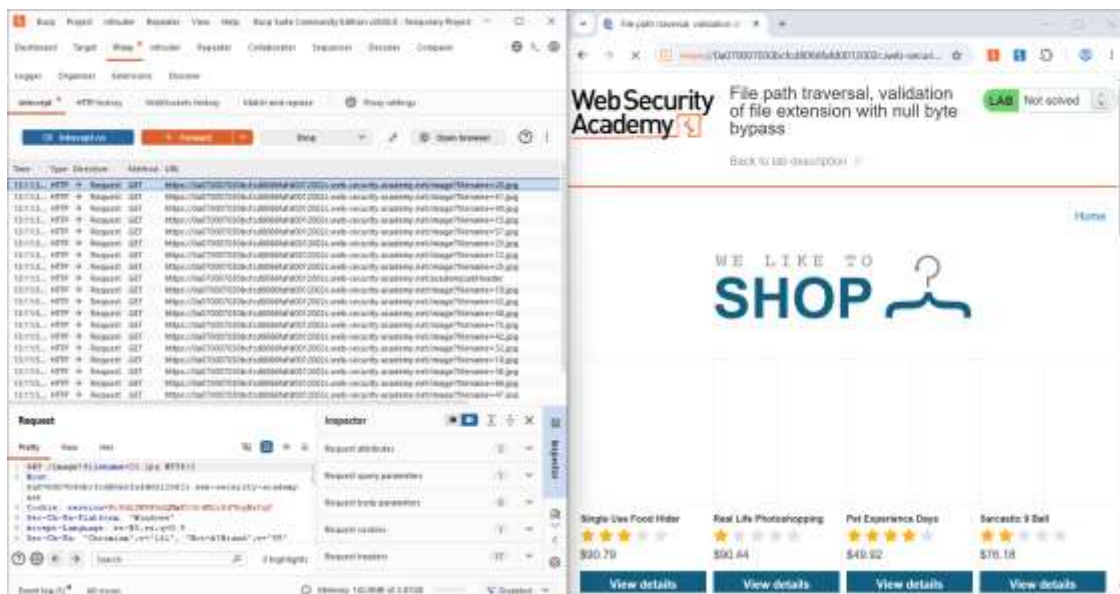


Figura 2. Historial de peticiones GET /image?filename=N.jpg en Burp Proxy sobre la tienda “We Like To Shop”.

Se seleccionó una de estas peticiones y se envió a Repeater mediante el menú contextual “Send to Repeater” para su análisis y modificación.

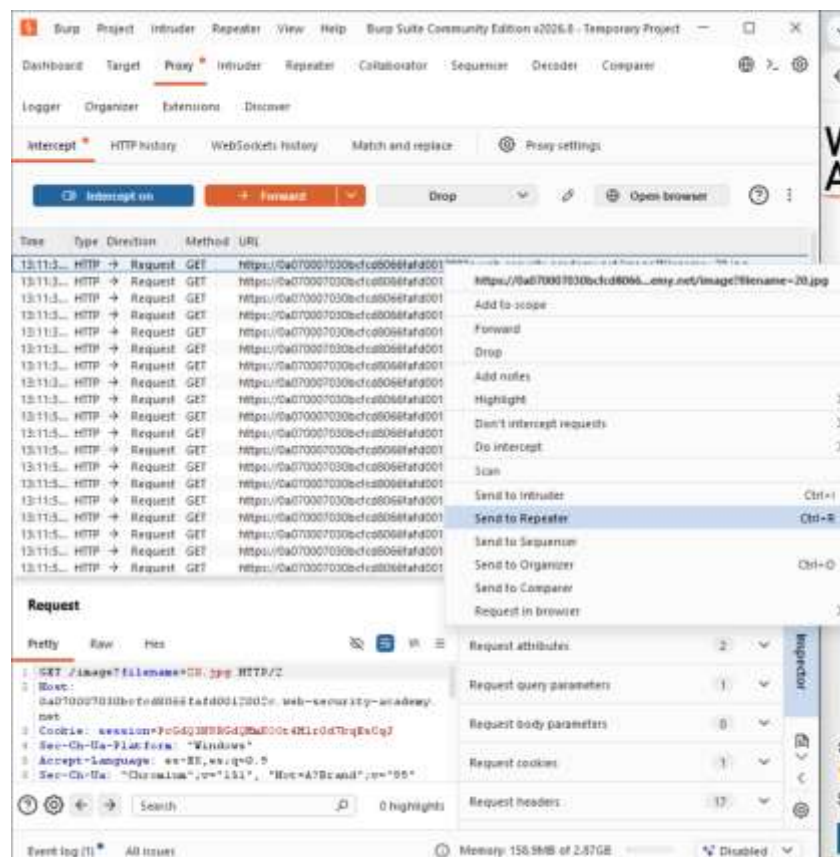


Figura 3. Envío de la petición seleccionada (filename=20.jpg) a Burp Repeater.

Dentro de Repeater se confirmó el valor original del parámetro filename antes de comenzar a modificarlo.

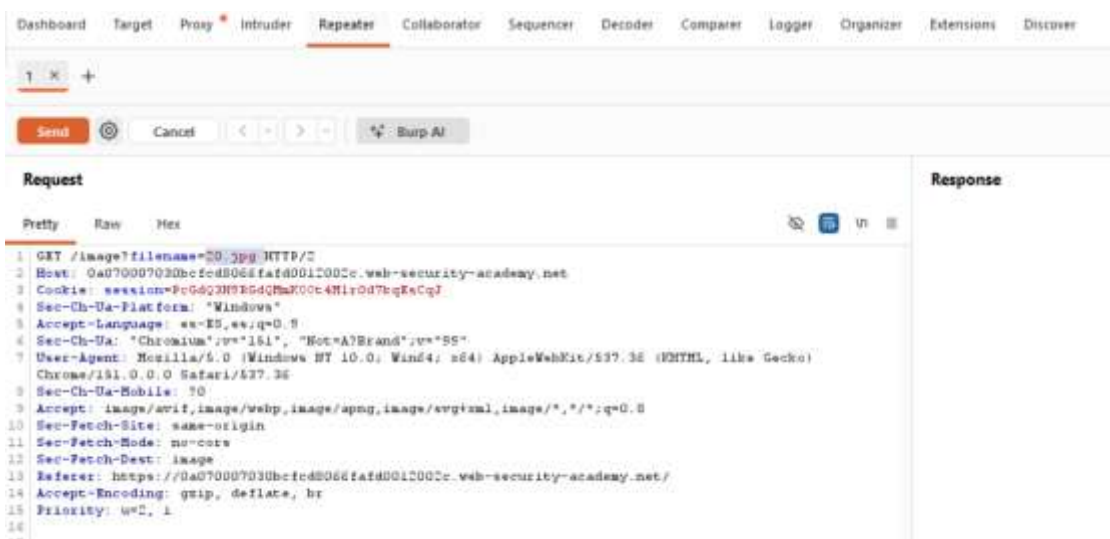


Figura 4. Petición original GET /image?filename=20.jpg visible en Repeater.

5. Fase 2: Construcción del payload con byte nulo

Un envío directo de una secuencia de traversal sin la extensión .png sería rechazado por la validación de la aplicación. Para evadirla, se construyó un payload que combina la secuencia de traversal, un byte nulo codificado en URL (%00) y la extensión exigida al final:

```
GET /image?filename=../../../../etc/passwd%00.png HTTP/2
```

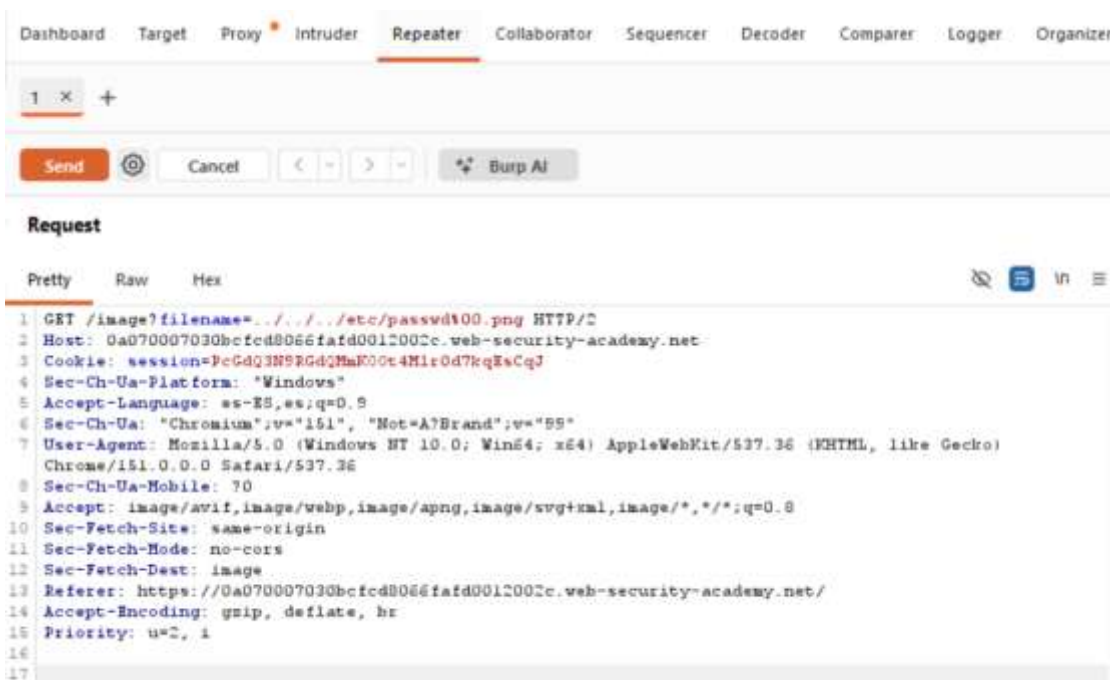


Figura 5. Petición modificada en Repeater con el payload ../../../../../../etc/passwd%00.png.

6. Fase 3: Explotación y confirmación

Al enviar la petición modificada, el servidor respondió con un código HTTP 200 OK. Cabe destacar que la cabecera Content-Type de la respuesta indicó image/png (la aplicación “creyó” que el archivo era una imagen PNG válida), pero el cuerpo de la respuesta contenía en realidad el contenido íntegro del archivo /etc/passwd.

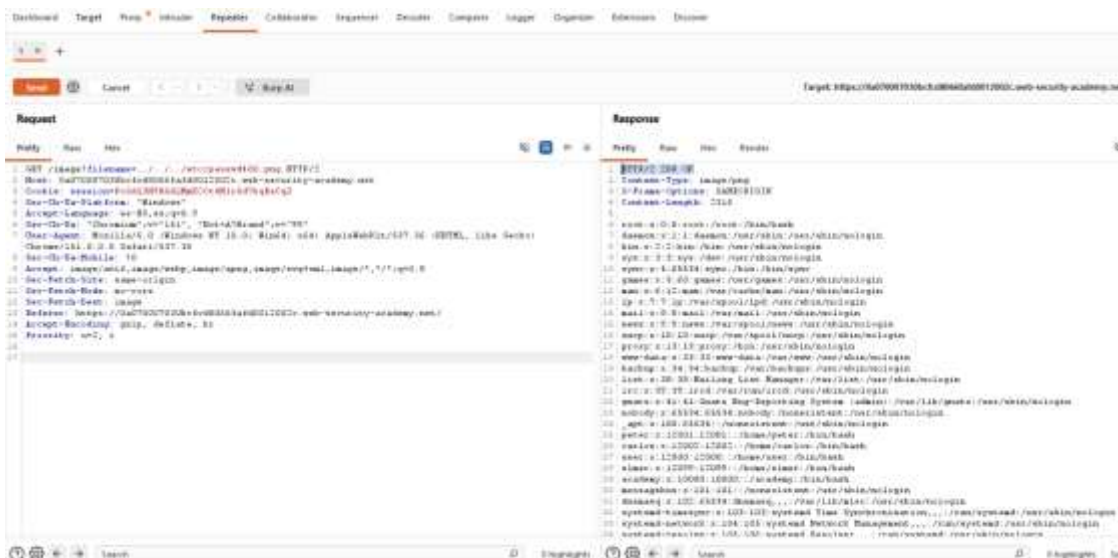


Figura 6. Respuesta del servidor con Content-Type: image/png pero con el contenido real de /etc/passwd.

6.1 Explicación del payload

El payload ../../etc/passwd%00.png se compone de tres partes con funciones diferenciadas:

- ../../etc/passwd: la secuencia de traversal clásica que, al resolverse, apunta al archivo objetivo en la raíz del sistema de archivos.
- %00: la codificación URL del byte nulo (0x00). El servidor web decodifica este valor a un byte nulo real antes de que la aplicación reciba la cadena.
- .png: la extensión exigida por la validación de la aplicación, colocada después del byte nulo para que la comprobación “termina en .png” se cumpla sobre la cadena completa, tal como la ve la capa de validación de alto nivel.

La clave del bypass está en que la capa de validación de la aplicación examina la cadena completa (../../etc/passwd\0.png) y confirma correctamente que termina en “.png”, dejándola pasar. Sin embargo, cuando esa misma cadena se entrega a una función de apertura de archivos de más bajo nivel (heredada de C), dicha función interpreta el byte nulo como el fin de la cadena y trunca todo lo que sigue, quedándose únicamente con ../../etc/passwd —sin la extensión .png que la validación había verificado.

Concepto teórico: null byte injection. Esta técnica, conocida como “inyección de byte nulo”, es un ejemplo clásico de discrepancia entre dos representaciones de una misma cadena de datos en distintas capas de un sistema. Fue especialmente común en aplicaciones que combinaban lenguajes de alto nivel, con bibliotecas subyacentes escritas en C (donde las cadenas siempre terminan en el primer byte nulo). Aunque los lenguajes y frameworks modernos han mitigado en gran medida este problema a nivel de las APIs estándar, sigue

siendo relevante en integraciones con bibliotecas nativas, código heredado o validaciones personalizadas mal implementadas.

6.2 Resolución del laboratorio

Al confirmar que la respuesta contenía el archivo `/etc/passwd` a pesar de la validación de extensión, PortSwigger marcó automáticamente el laboratorio como resuelto.

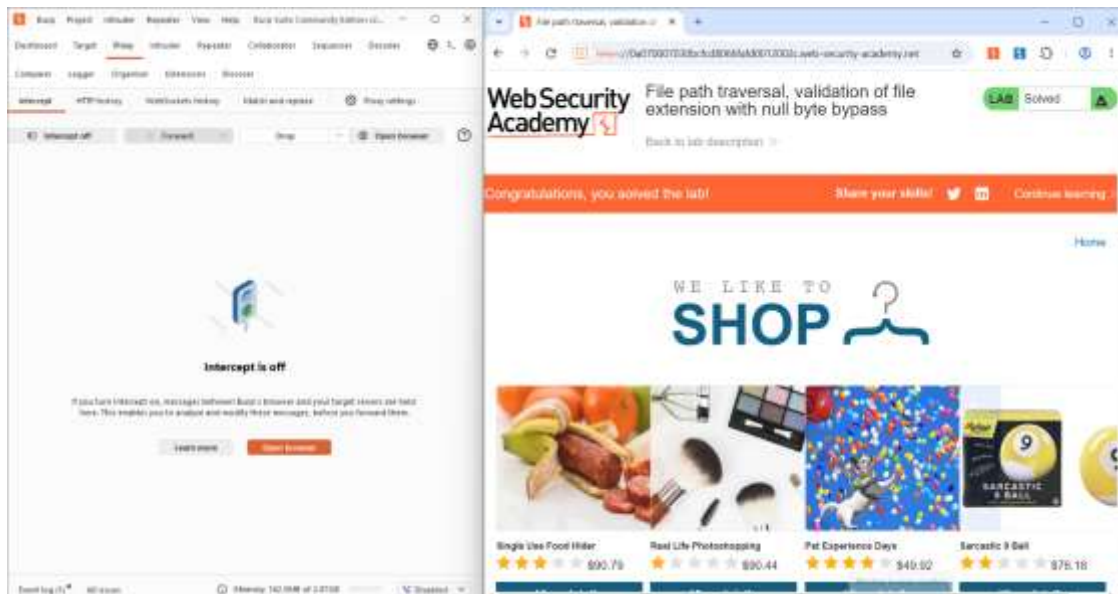


Figura 7. Confirmación de PortSwigger: “Congratulations, you solved the lab!”

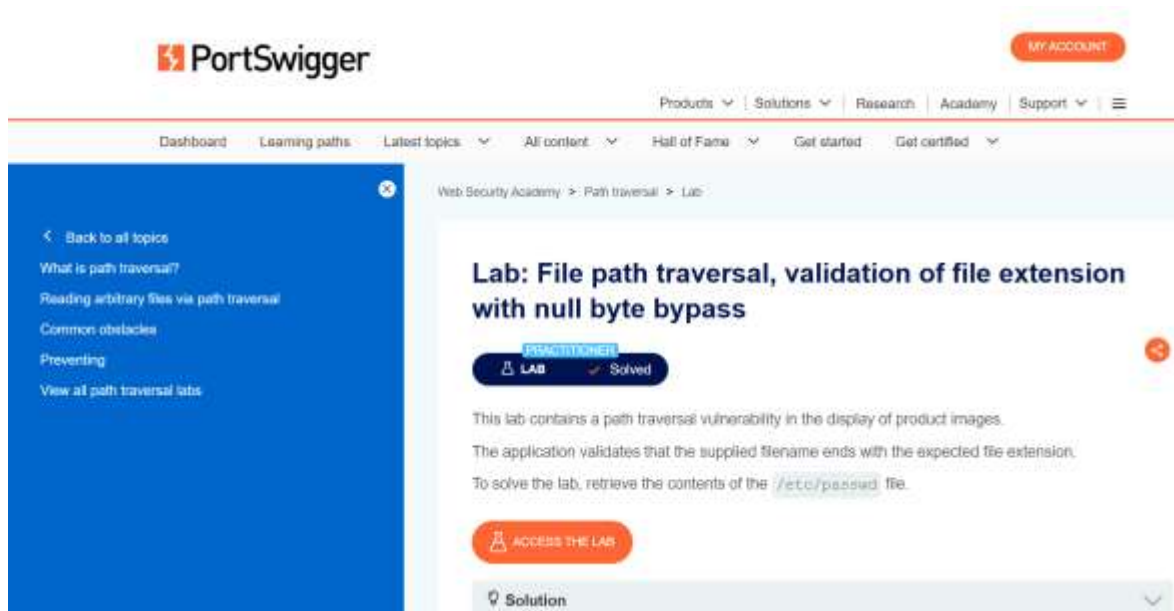


Figura 8. Estado final del laboratorio marcado como “Solved”.

7. Mitigación recomendada

Para prevenir esta clase de bypass basado en byte nulo, se recomienda:

- Rechazar explícitamente cualquier entrada que contenga bytes nulos (tanto el carácter literal como su codificación %00) antes de realizar cualquier otra validación.
- Utilizar APIs y frameworks modernos y actualizados para el manejo de archivos, que en su mayoría ya rechazan o filtran automáticamente los bytes nulos en rutas de archivo.
- Evitar delegar operaciones de archivo a bibliotecas nativas de bajo nivel sin revalidar la cadena resultante después de cualquier transformación o truncamiento posible.
- Validar la ruta final canonical (ya resuelta por el sistema operativo) contra el directorio base permitido, en lugar de basarse únicamente en comprobaciones de cadena como “termina en .png”.
- Sustituir el parámetro filename por un identificador indirecto validado contra una lista de archivos permitidos, eliminando por completo la necesidad de validar extensiones o rutas proporcionadas por el usuario.

8. Conclusiones

Este laboratorio, último de la serie dedicada a path traversal, ilustró cómo una validación adicional aparentemente sólida, puede resultar completamente ineficaz si no se considera cómo las distintas capas de un sistema interpretan los límites de una cadena de texto. El byte nulo actuó como un “separador invisible” para la capa de validación, pero como un terminador real para la capa de acceso a archivos.

En conjunto, los seis laboratorios de esta serie demostraron que las defensas contra path traversal deben aplicarse sobre la ruta ya normalizada y verificarse contra el directorio base permitido, en lugar de depender de comprobaciones parciales sobre la cadena de entrada sin procesar, sin importar cuántas capas de dichas comprobaciones se añadan.